DESENVOLVIMENTO DE APLICATIVOS MOBILE · MÓDULO 13

Desenvolvimento Multiplataforma

Flutter, React Native e Kotlin Multiplatform: arquitetura, código comparado e critérios de escolha

Prof. Dr. Francisco dos Santos Viana

São Luís – Maranhão – Brasil · 2026

Objetivos do módulo

Ao final deste módulo você será capaz de

- Explicar como Flutter, React Native e Kotlin Multiplatform chegam ao Android e ao iOS.
- Ler e escrever uma tela simples em Dart/Flutter e em TypeScript/React Native.
- Estruturar um projeto Kotlin Multiplatform com código comum e específico.
- Mapear os conceitos de Compose para os equivalentes em cada framework.
- Escolher a abordagem adequada a um projeto e a uma equipe.

Você já sabe mais do que imagina

Os três frameworks usam **UI declarativa**, **estado** e **componentes**, os mesmos conceitos dos Módulos 04–06.

Roteiro

Neste módulo

- Panorama
- Flutter
- React Native
- Kotlin Multiplatform
- Como escolher
- Fechamento

Panorama

Três estratégias de compartilhamento



Flutter

Compartilha **tudo**; a interface é desenhada pelo próprio motor, idêntica nas plataformas.

React Native

Compartilha **tudo**; os componentes viram widgets nativos de cada sistema.

KMP

Compartilha **o que fizer sentido**: só a lógica ou também a UI.

Comparação

	Flutter	React Native	Kotlin Multiplatform
Linguagem	Dart	JavaScript / TypeScript	Kotlin
Mantenedor	Google	Meta e comunidade	JetBrains (com apoio do Google)
Interface	widgets próprios desenhados pelo motor	componentes nativos	Compose Multiplatform ou nativa
Desenvolvimento	*hot reload*	*fast refresh*	*hot reload* (Compose)
Acesso nativo	*plugins* e *platform channels*	módulos nativos (Turbo Modules)	direto (`expect`/`actual`)
Outras plataformas	web, desktop	web (parcial)	desktop, web, servidor
Para quem sabe Kotlin	aprender Dart	aprender JS/TS e React	continua em Kotlin

Dica

Todos são usados em produção por grandes empresas. Escolha pelo **contexto** (equipe, código existente, requisitos), não por modismo.

Flutter

Flutter: tudo é widget

lib/painel.dart

```dart
class Painel extends StatefulWidget {
  const Painel({super.key});
  @override
  State<Painel> createState() => _PainelState();
}

class _PainelState extends State<Painel> {
  double temp = 24.0;                  // estado

  @override
  Widget build(BuildContext context) => Center(
    child: Column(mainAxisSize: MainAxisSize.min,
      children: [
        Text('${temp.toStringAsFixed(1)} °C'),
        ElevatedButton(
          onPressed: () => setState(() => temp += 0.5),
          child: const Text('Simular'),
        ),
      ]),
  );
}
```

Conceitos

- `StatelessWidget`: sem estado próprio.
- `StatefulWidget` + `State`: com estado; `setState` pede um novo `build`.
- Layout com widgets: `Column`, `Row`, `Padding`, `Center`.

Compose ↔ Flutter

`@Composable`	`Widget.build`
`mutableStateOf`	`setState`
`Modifier.padding`	`Padding(...)`
`LazyColumn`	`ListView.builder`
`ViewModel`	`ChangeNotifier`/Bloc

Flutter: ecossistema e estrutura

Começando

```
flutter create labmonitor_flutter
cd labmonitor_flutter
flutter run          # r = hot reload
flutter pub add http riverpod
flutter build appbundle
```

pubspec.yaml (trecho)

```
dependencies:
  flutter:
    sdk: flutter
  http: ^1.2.0
  flutter_riverpod: ^2.5.0
```

Dica

Pacotes em `https://pub.dev`. Confira as versões atuais antes de copiar.

Necessidade	Pacotes comuns
Estado	Provider, Riverpod, Bloc
Rede	`http`, `dio`
Banco local	`drift`, `sqflite`
Firebase	FlutterFire
BLE	`flutter_blue_plus`
MQTT	`mqtt_client`
Navegação	`go_router`

Pontos fortes e limites

UI idêntica em todas as plataformas, *hot reload* rápido e ótima documentação. O motor embutido deixa o APK maior que o nativo.

React Native

React Native: componentes e hooks

app/index.tsx (Expo)

```tsx
import { useState } from 'react';
import { View, Text, Button, StyleSheet } from 'react-native';

export default function Painel() {
  const [temp, setTemp] = useState(24.0);   // estado

  return (
    <View style={styles.centro}>
      <Text style={styles.valor}>{temp.toFixed(1)} °C</Text>
      <Button title="Simular"
        onPress={() => setTemp(t => t + 0.5)} />
    </View>
  );
}

const styles = StyleSheet.create({
  centro: { flex: 1, alignItems: 'center',
            justifyContent: 'center' },
  valor: { fontSize: 48, fontWeight: 'bold' },
});
```

Conceitos

- Componentes são funções que devolvem JSX.
- `useState`, `useEffect`: *hooks* (como `remember` e `LaunchedEffect`).
- Estilo parecido com CSS e layout *flexbox*.
- Nova arquitetura (Fabric, Turbo Modules, JSI).

Expo

Caminho recomendado para começar: `npx create-expo-app`, rotas por arquivos e o app *Expo Go*.

Kotlin Multiplatform

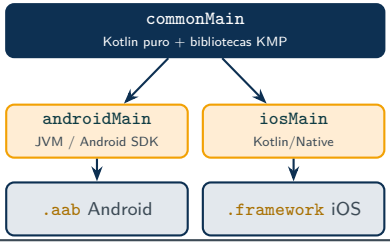
Estrutura de um projeto KMP

Projeto KMP

```
labmonitor-kmp/
|-- composeApp/
|   `-- src/
|       |-- commonMain/    UI Compose comum
|       |-- androidMain/   MainActivity
|       `-- iosMain/       MainViewController
|-- shared/
|   `-- src/
|       |-- commonMain/    modelos, repositórios,
|       |                  casos de uso, Ktor
|       |-- androidMain/   implementações Android
|       `-- iosMain/       implementações iOS
`-- iosApp/                projeto Xcode (Swift)
```

Dica

O **Kotlin Multiplatform Wizard** (`https://kmp.jetbrains.com`) gera essa estrutura pronta.



Camada	Bibliotecas KMP
Rede	Ktor Client + kotlinx.serialization
Banco	Room (KMP), SQLDelight, DataStore
Estado	ViewModel (lifecycle KMP), coroutines
DI	Koin
UI	Compose Multiplatform

expect e actual

commonMain/Plataforma.kt

```
expect fun nomePlataforma(): String
```

androidMain/Plataforma.android.kt

```
actual fun nomePlataforma() =
  "Android ${Build.VERSION.SDK_INT}"
```

iosMain/Plataforma.ios.kt

```
import platform.UIKit.UIDevice

actual fun nomePlataforma() =
  UIDevice.currentDevice.systemName() + " " +
  UIDevice.currentDevice.systemVersion
```

commonMain: uso

```
Text("Rodando em ${nomePlataforma()}")
```

O mecanismo

`expect` declara **o quê** no código comum; cada plataforma fornece o `actual`, **como**, usando suas APIs nativas (Android SDK ou frameworks da Apple).

Prefira interfaces

Para dependências maiores (BLE, sensores), declare uma `interface` no código comum e injete implementações por plataforma com Koin. Fica mais fácil de testar.

Atenção

Compilar e rodar a parte iOS exige **macOS com Xcode**.

Compartilhando a lógica do LabMonitor

shared/commonMain/ClimaRepository.kt

```
class ClimaRepository(private val http: HttpClient) {
  suspend fun atual(lat: Double, lon: Double): ClimaDto =
    http.get("https://api.open-meteo.com/v1/forecast") {
      parameter("latitude", lat)
      parameter("longitude", lon)
      parameter("current", "temperature_2m")
    }.body()
}

class ClimaViewModel(
  private val repo: ClimaRepository
) : ViewModel() {                  // lifecycle KMP
  val estado = MutableStateFlow<Double?>(null)
  fun carregar() = viewModelScope.launch {
    estado.value = repo.atual(-2.53, -44.30)
      .current.temperatura
  }
}
```

O que mudou em relação ao Android

Quase nada: Retrofit → **Ktor**, Hilt → **Koin**. DTOs, `Flow`, `ViewModel`, casos de uso e testes continuam iguais.

Migração gradual

Um app Android existente pode mover **primeiro** os modelos e casos de uso para um módulo `shared`, sem reescrever a UI. É a forma mais segura de adotar KMP.

Como escolher

A mesma tela nos três mundos

Conceito	Compose (Kotlin)	Flutter (Dart)	React Native (TS)
Unidade de UI	função `@Composable`	classe `Widget`	função componente
Estado local	`remember { mutableStateOf() }`	`State` + `setState`	`useState`
Efeito ao entrar	`LaunchedEffect`	`initState`	`useEffect`
Lista longa	`LazyColumn`	`ListView.builder`	`FlatList`
Layout	`Column`/`Row`/`Box`	`Column`/`Row`/`Stack`	`View` + flexbox
Estado da tela	`ViewModel` + `StateFlow`	Riverpod / Bloc	Context, Zustand, Redux
Assíncrono	coroutines / `Flow`	`Future` / `Stream`	`Promise` / `async`

Os conceitos se transferem: quem domina um framework declarativo aprende o outro em semanas.

Critérios de decisão

Perguntas a fazer

1. Precisa de iOS? Quando?
2. Que linguagem a equipe já domina?
3. Há código Android/Kotlin existente?
4. O app depende muito de hardware (BLE, câmera, sensores)?
5. A UI deve ser idêntica ou seguir o padrão de cada sistema?
6. Há também web ou desktop no horizonte?

Cenário	Sugestão
Só Android, hardware intenso	Kotlin nativo
App Android existente indo para iOS	KMP
Equipe web/React	React Native
UI de marca idêntica em tudo, equipe nova	Flutter
Protótipo rápido para as duas lojas	Flutter ou RN + Expo

Para o LabMonitor

Como o app usa BLE e já está em Kotlin, o caminho natural para o iOS é **KMP**: compartilhar modelos, repositórios e casos de uso, e escolher entre Compose Multiplatform e SwiftUI para a UI.

Fechamento

Resumo

Resumo

- Flutter desenha a própria UI com Dart; React Native usa componentes nativos com JS/TS; KMP compartilha Kotlin e deixa a UI a critério.
- Os três são declarativos: estado, componentes e efeitos se mapeiam diretamente para Compose.
- KMP usa `commonMain` + `expect`/`actual` e bibliotecas como Ktor, Room, Koin e Compose Multiplatform.
- iOS sempre exige macOS e Xcode para compilar e publicar.
- A escolha depende da equipe, do código existente, do hardware e das plataformas-alvo.

Exercícios

Exercício 1 — Flutter

Reescreva em Flutter a tela de temperatura da Open-Meteo (Módulo 08) com `http` e um `FutureBuilder` mostrando carregando, sucesso e erro.

Exercício 2 — React Native

Crie com Expo uma lista de equipamentos com `FlatList` e uma tela de detalhe usando o roteamento por arquivos.

Exercício 3 — Comparação

Meça, para o app do exercício 1 e para o equivalente em Compose, o tamanho do APK release e o tempo de abertura. Discuta os resultados.

LabMonitor — Etapa 13

Extraia modelos, DTOs e o `VerificarAlertasUseCase` para um módulo `shared` KMP com testes em `commonTest`. Opcional: tela de painel em Compose Multiplatform rodando no desktop.

Para aprofundar

- Flutter: `https://docs.flutter.dev` Flutter para devs Android: `https://docs.flutter.dev/get-started/flutter-for/android-devs`
- React Native: `https://reactnative.dev` Expo: `https://docs.expo.dev`
- Kotlin Multiplatform: `https://kotlinlang.org/docs/multiplatform.html`
- KMP no Android: `https://developer.android.com/kotlin/multiplatform`
- Compose Multiplatform: `https://www.jetbrains.com/compose-multiplatform/`
- Ktor Client: `https://ktor.io/docs/client-create-new-application.html`

Fim do Módulo 13

Próximo: Módulo 14 — Tópicos avançados e projeto final: IA no dispositivo, segurança, monetização e carreira.